

JANATICS DATA ENGINE

System & Development Design Document

A to Z Architecture Reference · v1.0

Table of Contents

1. Executive Summary

Janatics DataEngine is a provider-agnostic, registry-driven data access class library built on .NET 8 that exposes two unified API contracts — /fetch and /forge — to any consuming .NET application. It eliminates per-entity CRUD boilerplate, supports runtime multi-database routing, and ships a companion React admin suite for registry governance and query composition.

1.1 Problem Statement

- Every new ERP table required a new Repository, Service, Controller, and DTO class — 4 files per entity, hundreds of files across the system
- Switching between Oracle ERP, SQL Server HR, and SQLite Dev required manual connection string management scattered across multiple projects
- No centralised governance of which tables exist in which database, what their primary keys are, or who is authorised to write to them
- Frontend developers had no visual tooling to build queries or inspect registered entities; all payload construction was done by hand
- SQL dialect differences (Oracle ROWNUM/FETCH, SQL Server OFFSET/FETCH, parameter prefix : vs @) were re-implemented ad hoc in every project

1.2 Solution Summary

DataEngine packages a central Entity & DB Registry, a runtime connection factory, a dialect-aware SQL builder, and two service endpoints into a single NuGet-distributed DLL. Any Janatics portal adds three lines to Program.cs and immediately gains fully parameterised, paginated, multi-database read/write without writing table-specific code.

1.3 Business Value

2. Architecture Overview

2.1 High-Level System Map



■ JANATICS PORTAL ECOSYSTEM ■



■ React Frontend (Vite + TypeScript + Ant Design) ■

The Registry is the brain of DataEngine. It is stored in a dedicated metadata database (MetaDB — typically a lightweight SQL Server or SQLite instance) and is independent of any business database.

3.1 Entity-Relationship Overview

DbProfiles EntityRegistry ColumnRegistry



ProfileId PK EntityId PK ColumnId PK
ProfileName ProfileId FK → DbProfiles EntityId FK → EntityReg
Provider EntityName ColumnName
Host SchemaName DataType
Port PkColumn IsRequired
Database IsReadOnly IsPrimaryKey
Username AllowedRoles (CSV) IsForeignKey
PasswordHash SoftDeleteColumn FkReference
Status CreatedAt CreatedAt
CreatedAt CreatedBy
UpdatedAt
AuditLog



LogId PK
ProfileId FK (nullable)
EntityId FK (nullable)
Action (ADD_PROFILE / EDIT_ENTITY / DELETE_COLUMN ...)
ChangedBy
ChangedAt
OldValue JSON snapshot
NewValue JSON snapshot

3.2 DbProfiles Table

3.3 EntityRegistry Table

3.4 ColumnRegistry Table

4. Janatics.DataEngine — Class Library Structure

4.1 Project Layout

Janatics.DataEngine/ ← .NET 8 Class Library project



- Models/ ← Public API contracts (shared with consumers)
- ■■■■ FetchRequest.cs ← /fetch payload model
- ■■■■ FetchResult.cs ← Paginated response wrapper
- ■■■■ ForgeRequest.cs ← /forge payload model
- ■■■■ ForgeResult.cs ← CUD response with affected ID
- ■■■■ FilterCondition.cs ← Column + operator + value
- ■■■■ SearchOptions.cs ← Full-text search config
- ■■■■ SortOptions.cs ← Column + direction
- ■■■■ NodeRequest.cs ← Child join node (fetch)
- ■■■■ NodeForge.cs ← Child write node (forge)



- Registry/ ← Entity & DB metadata resolution

■ ■■■■ *Models/*

- ■ ■■■■ DbProfile.cs
- ■ ■■■■ EntityMeta.cs
- ■ ■■■■ ColumnMeta.cs
- ■■■■ IEntityRegistryService.cs
- ■■■■ EntityRegistryService.cs ← Queries MetaDB, maps entity → profile
- ■■■■ EntityRegistryCache.cs ← IMemoryCache wrapper, 5-min TTL



- Infrastructure/ ← DB connectivity
- ■■■■ IDbConnectionFactory.cs
- ■■■■ DbConnectionFactory.cs ← Resolves provider, builds IDbConnection
- ■■■■ SqlDialect.cs ← Param prefix, pagination clause, quoting
- ■■■■ ProviderDetector.cs ← Auto-sniff from connection string



- Builders/ ← SQL generation (pure, stateless)
- ■■■■ FetchSqlBuilder.cs ← SELECT with filter/sort/page/join
- ■■■■ ForgeSqlBuilder.cs ← INSERT / UPDATE / soft-DELETE
- ■■■■ WhitelistValidator.cs ← Entity + column name validation



- Services/ ← Business orchestration
- ■■■■ IFetchService.cs
- ■■■■ FetchService.cs ← Orchestrates /fetch end-to-end
- ■■■■ IForgeService.cs
- ■■■■ ForgeService.cs ← Orchestrates /forge end-to-end



- Exceptions/ ← Typed error hierarchy

- ■■■■ DataEngineException.cs ← Base
- ■■■■ EntityNotFoundException.cs
- ■■■■ ReadOnlyViolationException.cs
- ■■■■ RoleViolationException.cs
- ■■■■ SqlBuildException.cs

■

■■■ *Extensions/*

■■■ ServiceCollectionExtensions.cs ← AddDataEngine(services, config)

4.2 Consumer Integration

Any portal references the DLL and registers it in three lines:

// Program.cs

builder.Services.AddDataEngine(builder.Configuration);

// appsettings.json (minimum config)

"DataEngine": {

"MetaDb": {

"Provider": "SqlServer",

"ConnectionString": "Server=metadb;Database=DataEngineRegistry;..."

},

"CacheTtlMinutes": 5,

"EnableAuditLog": true

}

// DataController.cs (minimal — consumer writes this)

[HttpPost("/fetch")]

public Task<ActionResult> Fetch([FromBody] FetchRequest req)

=> _fetchService.ExecuteAsync(req, User);

[HttpPost("/forge")]

public Task<ActionResult> Forge([FromBody] ForgeRequest req)

=> _forgeService.ExecuteAsync(req, User);

5. API Contracts

5.1 POST /fetch — Read Operation

The /fetch endpoint accepts a structured query descriptor and returns a paginated, optionally counted result set. It supports column projection, multi-condition filtering, full-text search, sorting, and child table joins — all driven by a single JSON payload.

5.1.1 Request Schema

5.1.2 FilterCondition Object

5.1.3 Response Schema

5.1.4 Full Example

```
{
  "rootEntity": "PurchaseOrder",
  "page": 2,
  "pageSize": 25,
  "includeCount": true,
  "select": ["Id", "PONumber", "Status", "TotalAmount", "OrderDate"],
  "filters": [
    { "column": "Status", "op": "eq", "value": "Approved" },
    { "column": "TotalAmount", "op": "gte", "value": 5000 }
  ],
  "search": { "term": "PO-2026", "columns": ["PONumber"] },
  "sort": { "column": "OrderDate", "direction": "desc" },
  "nodes": [
    { "nodeEntity": "PurchaseOrderLine", "parentLink": "PurchaseOrderId" }
  ]
}
```

5.2 POST /forge — Write Operation

The /forge endpoint handles CREATE, UPDATE, and soft-DELETE in a single transactional call. Parent and all child nodes commit atomically; any failure rolls back the entire transaction.

5.2.1 Request Schema

5.2.2 NodeForge Object

5.2.3 Operation Behaviour Matrix

6. Runtime Execution Flow

6.1 /fetch End-to-End

HTTP POST /fetch { rootEntity: "PurchaseOrder", filters: [...], page: 2 }



[1] DataController.Fetch(req, User)



[2] FetchService.ExecuteAsync(req, userPrincipal)



■■■■■ [3] EntityRegistryService.LookupAsync("PurchaseOrder")

■ ■■■■ Check IMemoryCache (hit → skip DB)

■ ■■■■ Cache miss → SELECT from EntityRegistry MetaDB

■ Returns: { ProfileId, SchemaName:"JANATICS",

■ PkColumn:"Id", AllowedRoles, IsReadOnly }



■■■■■ [4] WhitelistValidator.Validate(req, entityMeta)

■ ■■■■ Entity registered?

■ ■■■■ Requested columns in ColumnRegistry?

■ ■■■■ Filter columns in ColumnRegistry?

■ ■■■■ User role in AllowedRoles?

■ Throws RoleViolationException or SqlBuildException



■■■■■ [5] ConnectionFactory.Create(profileId)

■ ■■■■ Resolve DbProfile from cache/registry

■ ■■■■ Decrypt PasswordHash

■ ■■■■ Return OracleConnection | SqlConnection | SqliteConnection



■■■■■ [6] FetchSqlBuilder.Build(req, entityMeta, dialect)

■ ■■■■ Build SELECT clause (all cols or projected)

■ ■■■■ Build WHERE clause (AND-chained filter conditions)

■ ■■■■ Build LIKE clauses for search term

■ ■■■■ Build JOIN clauses for nodes

■ ■■■■ Build ORDER BY

■ ■■■■ Wrap in provider-specific pagination:

■ Oracle: OFFSET :skip ROWS FETCH NEXT :take ROWS ONLY

■ SqlServer: OFFSET @skip ROWS FETCH NEXT @take ROWS ONLY

■ SQLite: LIMIT :take OFFSET :skip

■

■■■■■ [7] `Dapper.QueryAsync(sql, parameters, connection)`

■ Optional: `Dapper.ExecuteScalarAsync` for `COUNT(*)`

■

■■■■■ [8] Build `FetchResult { success, totalCount, page, data, nodes }`

■■■■ Emit Serilog structured event

`{ entity, profile, rows, executionMs, page }`

6.2 /forge End-to-End

HTTP POST /forge { rootEntity:"PurchaseOrder", operation:"create", content:{...} }

■

▼

[1] `ForgeService.ExecuteAsync(req, userPrincipal)`

■

■■■■■ [2] `EntityRegistryService.LookupAsync("PurchaseOrder")`

■ Same cache/registry lookup as /fetch

■

■■■■■ [3] `WhitelistValidator.ValidateForge(req, entityMeta, columnRegistry)`

■ ■■■■ IsReadOnly? → `ReadOnlyViolationException`

■ ■■■■ User role in `AllowedRoles`?

■ ■■■■ All content keys in `ColumnRegistry`?

■ ■■■■ Required columns present for CREATE?

■

■■■■■ [4] `ConnectionFactory.Create(profileId)`

■

■■■■■ [5] `Open IDbConnection → BeginTransaction()`

■

■■■■■ [6] `ForgeSqlBuilder.BuildParent(req, entityMeta, dialect)`

■ ■■■■ CREATE: `INSERT INTO JANATICS.PurchaseOrder (...) VALUES (...)`

■ ■ + returning clause → capture new PK

■ ■■■■ UPDATE: `UPDATE JANATICS.PurchaseOrder SET ... WHERE Id = :rootId`

■ ■■■■ DELETE: `UPDATE JANATICS.PurchaseOrder`

■ `SET IsDeleted = 1 WHERE Id = :rootId`

■

■■■■ [7] `Dapper.ExecuteAsync (parent SQL, tx)`

■

■■■■ [8] For each node in `req.Nodes`:

■ ■■■ Inject `parentLink FK = newParentId`

■ ■■■ `ForgeSqlBuilder.BuildNode(node, dialect)`

■ ■■■ `Dapper.ExecuteAsync (node SQL, tx)`

■

■■■■ [9] `tx.Commit()`

■

■■■■ [Err] `tx.Rollback()` on any exception

■

■■■■ [10] Return `ForgeResult { success, entityId, operation, affectedNodes }`

7. SQL Builder Design

7.1 Dialect Abstraction

`SqlDialect` is a value type that encapsulates all provider-specific SQL differences. `FetchSqlBuilder` and `ForgeSqlBuilder` receive a dialect instance and call its methods — they never branch on provider type directly.

7.2 FetchSqlBuilder Logic

`FetchSqlBuilder.Build()` is a pure function — same inputs always produce same SQL. It operates in five stages:

SELECT clause: if `req.Select` is empty → `SELECT *` else `SELECT col1, col2 ...` (schema-qualified table alias)

WHERE clause: each `FilterCondition` mapped to parameterised predicate; parameters named `p0, p1 ... pN`; LIKE values auto-wrapped with `%`

SEARCH clause: if search term present, OR-chain across specified columns using provider `LOWER()` for case-insensitive match

JOIN clause: each node produces `LEFT JOIN {schema}.{nodeEntity} n0 ON n0.{parentLink} = t.{pk}`

SORT + PAGINATION: `ORDER BY` appended, then `dialect.PaginationClause(skip, take)` wraps the whole query

Security Note: Column and table names are interpolated into SQL strings after passing `WhitelistValidator` — they come from the registry, not raw user input. Parameter values are always Dapper parameters, never concatenated.

7.3 ForgeSqlBuilder Logic

`operation = "create"`

→ `INSERT INTO {schema}.{table} ({col1},{col2},...)`

`VALUES ({p0},{p1},...)`

[+ dialect returning clause]

`operation = "update"`

→ `UPDATE {schema}.{table}`

SET {col1} = {p0}, {col2} = {p1}, ...

WHERE {pkColumn} = {pId}

operation = "delete" (soft)

→ UPDATE {schema}.{table}

SET {softDeleteColumn} = {softDeleteValue}

WHERE {pkColumn} = {pId}

child node (same builder, per node in req.Nodes):

→ parentLink FK auto-injected into content dict

→ same create/update logic applied

8. Entity Registry Service & Caching

8.1 Lookup Lifecycle

EntityRegistryService.LookupAsync("PurchaseOrder")

■

■■■■ IMemoryCache.TryGetValue("entity:PurchaseOrder")

■ HIT → return cached EntityMeta (zero DB calls)

■ MISS → continue

■

■■■■ SELECT e.*, p.Provider, p.PasswordHash

■ FROM EntityRegistry e

■ JOIN DbProfiles p ON p.ProfileId = e.ProfileId

■ WHERE e.EntityName = @name

■

■■■■ Decrypt PasswordHash → ConnectionString

■

■■■■ Load ColumnRegistry for entity (separate SELECT)

■

■■■■ IMemoryCache.Set("entity:PurchaseOrder", meta,

■ absoluteExpiration: now + CacheTtlMinutes)

■

■■■■ return EntityMeta

8.2 Cache Invalidation Strategy

8.3 ConnectionFactory

ConnectionFactory holds a dictionary of ProfileId → decryptedConnectionString. It is registered as a singleton and populated at startup from all active DbProfiles. It returns a new IDbConnection per call (connections are not pooled at this layer — ADO.NET connection pool handles physical connection reuse).

9. Security Design

9.1 Threat Model

9.2 Role Enforcement Flow

Request arrives with JWT (or Windows Identity)

■

■■■ ClaimsPrincipal.GetRoles() → ["user", "hr"]

■

■■■ EntityMeta.AllowedRoles = ["admin", "hr"]

■

■■■ Intersection: ["hr"] → non-empty → ALLOW

■

■■■ Empty intersection → throw RoleViolationException(403)

9.3 Connection String Security

Passwords stored via AES-256-CBC with a machine-level key from environment variable
DATA_ENGINE_CRYPTO_KEY

PasswordHash column contains: base64(IV + ciphertext)

Key never stored in DB; rotated per environment (dev / staging / prod)

Connection strings logged with password replaced by ***** in Serilog output

10. Frontend Admin Suite

10.1 Component Map

src/

■■■ pages/

■ ■■■ RegistryManager.tsx ← DB Profiles + Entity list + Column panel

■ ■■■ QueryBuilder.tsx ← /fetch visual query composer

■ ■■■ FieldMapper.tsx ← /forge payload builder

■

■■■ components/

- ■■■ registry/
 - ■ ■■■ ProfileCard.tsx
 - ■ ■■■ ProfileForm.tsx
 - ■ ■■■ EntityCard.tsx
 - ■ ■■■ EntityForm.tsx
 - ■ ■■■ ColumnPanel.tsx
- ■■■ querybuilder/
 - ■ ■■■ FilterRow.tsx
 - ■ ■■■ ColumnSelector.tsx
 - ■ ■■■ SortConfig.tsx
 - ■ ■■■ PaginationConfig.tsx
 - ■ ■■■ ResultsTable.tsx
- ■■■ fieldmapper/
 - ■■■ ColumnRow.tsx
 - ■■■ ChildSection.tsx
-
- services/
 - ■■■ registryApi.ts ← CRUD for profiles + entities + columns
 - ■■■ fetchApi.ts ← POST /fetch
 - ■■■ forgeApi.ts ← POST /forge
-
- types/
 - FetchRequest.ts
 - ForgeRequest.ts
 - Registry.ts

10.2 Page Descriptions

10.3 Registry Manager Features

Add, edit, delete DB profiles with auto-generated connection string preview

Test Connection button — calls dedicated endpoint that attempts a real DB ping

Auto-Discover tables — queries INFORMATION_SCHEMA / ALL_TABLES and lists unregistered tables for import

Expandable entity cards showing column registry with type badges and FK annotations

Column panel with add/remove and full type + constraint configuration

Audit log viewer — filter by entity, action type, or date range

10.4 Query Builder Features

Entity selector driven by registry — only registered entities shown

Column selection with type chips and all-or-none shortcuts

Multi-condition filter builder with operator dropdown scoped by column data type

Full-text search with per-column scope selection

Sort configuration with column selector and ASC/DESC toggle

Child join (node) configuration with FK column override
Pagination with preset page-size buttons (10 / 25 / 50 / 100 / 500)
Live JSON payload preview with copy-to-clipboard
Execute against real API with inline result table and prev/next navigation

10.5 Field Mapper Features

INSERT / UPDATE mode toggle — UPDATE reveals Transaction ID input
Column rows auto-generated from ColumnRegistry — required fields marked, data types respected
Progress bar showing mapped / total columns
Child entity sections with FK column config and Record ID for child-level update mode
Payload preview tab with validation summary (missing required fields listed)
Submit wired to /forge with success/error state

11. Error Handling & Result Pattern

11.1 Exception Hierarchy

DataEngineException (base)
■■■ EntityNotFoundException → HTTP 404 (entity not in registry)
■■■ ReadOnlyViolationException → HTTP 403 (forge on read-only entity)
■■■ RoleViolationException → HTTP 403 (user role not in AllowedRoles)
■■■ SqlBuildException → HTTP 400 (unknown column / invalid filter)
■■■ ConnectionException → HTTP 503 (DB unreachable)

11.2 Result Types

```
public record FetchResult<T>(  
    bool Success,  
    IReadOnlyList<T>? Data,  
    int? TotalCount,  
    int Page,  
    int PageSize,  
    string? Error,  
    FetchMeta Meta
```

```
);
```

```
public record ForgeResult(  
    bool Success,  
    string? EntityId,  
    string Operation,  
    int AffectedNodes,  
    string? Error,  
    ForgeMeta Meta
```

```
);
```

11.3 Global Exception Middleware

The consuming portal registers DataEngine's exception middleware which maps DataEngineException subtypes to appropriate HTTP status codes and a consistent error envelope:

```
{  
  "success": false,  
  "error": "Entity 'UnknownTable' is not registered in DataEngine registry.",  
  "errorCode": "ENTITY_NOT_FOUND",  
  "traceId": "00-4bf5122f-1a2b3c4d-01"  
}
```

12. Observability & Logging

12.1 Structured Log Events

12.2 Serilog Configuration

```
Log.Logger = new LoggerConfiguration()  
  .Enrich.FromLogContext()  
  .Enrich.WithProperty("Application", "DataEngine")  
  .WriteTo.Console(outputTemplate:  
    "[{Timestamp:HH:mm:ss} {Level:u3}] {Message:l} {Properties}{NewLine}{Exception}")  
  .WriteTo.Seq("http://seq.janatics.local")  
  .MinimumLevel.Override("Janatics.DataEngine", LogEventLevel.Debug)  
  .CreateLogger();
```

12.3 Performance Monitoring

Every service method uses System.Diagnostics.Stopwatch to capture executionMs

Thresholds: >500ms logs Warning; >2000ms logs Error with full SQL in Development environments

MetricsRecorder (optional) emits counters to Application Insights or Prometheus via OpenTelemetry

13. Testing Strategy

13.1 Test Pyramid

13.2 Key Unit Test Cases

FetchSqlBuilder produces correct Oracle FETCH NEXT pagination clause

FetchSqlBuilder produces correct OFFSET/FETCH for SQL Server

FilterCondition "between" produces BETWEEN :p0 AND :p1 with both parameter values

FilterCondition "contains" wraps value in % on both sides
WhitelistValidator rejects unknown entity name with EntityNotFoundException
WhitelistValidator rejects column not in ColumnRegistry with SqlBuildException
WhitelistValidator rejects user with no matching role with RoleViolationException
ForgeSqlBuilder for "delete" produces UPDATE ... SET IsDeleted = 1 not DELETE FROM
EntityRegistryCache returns cached value on second call without hitting DB

13.3 Integration Test Setup

```
public class FetchServiceIntegrationTests : IAsyncLifetime {  
    private SqliteConnection _metaConn;  
    private SqliteConnection _dataConn;  
  
    public async Task InitializeAsync() {  
        _metaConn = new SqliteConnection("Data Source=:memory:");  
        await SeedRegistryAsync(_metaConn); // DbProfile + EntityMeta  
        _dataConn = new SqliteConnection("Data Source=:memory:");  
        await SeedDataAsync(_dataConn); // Actual table + rows  
    }  
}
```

[Fact]

```
public async Task Fetch_WithPagination_ReturnsCorrectPage() { ... }  
  
}
```

14. Deployment & DevOps

14.1 NuGet Package

DataEngine is published as Janatics.DataEngine.nupkg to an internal NuGet feed (Azure Artifacts or JFrog)
Versioning: SemVer 2.0 — MAJOR.MINOR.PATCH (e.g., 1.0.0 → 1.1.0 for new features, 2.0.0 for breaking API changes)
Consumers pin to a minor version range: <PackageReference Include="Janatics.DataEngine" Version="[1.0,2.0)" />
CHANGELOG.md maintained per release with migration notes for any breaking changes

14.2 Environment Configuration

14.3 CI/CD Pipeline

Git Push → GitHub / Azure DevOps



Build Stage

- dotnet restore
- dotnet build --configuration Release
- dotnet test (unit + integration)



Pack Stage

- dotnet pack --configuration Release
- Publish to internal NuGet feed



Frontend Build

- npm ci
- npm run type-check
- npm run lint
- npm run test (Vitest)
- npm run build → dist/



Deploy Stage (Staging)

- Run EF migrations on MetaDB
- Deploy API host (IIS / Docker)
- Deploy React dist/ → Nginx / Azure Static Web Apps

15. Development Roadmap

Phase 1 — Foundation (Weeks 1–3)

Phase 2 — Registry & Connectivity (Weeks 4–5)

Phase 3 — Frontend Admin Suite (Weeks 6–8)

Phase 4 — Security & Hardening (Week 9)

16. Architecture Decisions & Constraints

16.1 Architecture Decision Records

ADR-001: Dapper over Entity Framework Core

ADR-002: Registry in MetaDB over appsettings.json

ADR-003: Soft Delete Only

16.2 Known Constraints

Table name collisions across multiple DBs (same name in Oracle and SQL Server) require unique entity aliases in registry; DataEngine does not auto-resolve collisions

Stored procedure calls are out of scope for v1.0 — DataEngine executes dynamic SQL only

Oracle RETURNING INTO requires OracleParameter with output direction — this is encapsulated in OracleDialect.GetNewIdClause() and is the only provider-specific code path in ForgeService

pageSize is capped at 1000 per request — bulk exports must paginate in the caller

Complex joins (multi-level grandchild) are deferred to v2.0; v1.0 supports one level of child nodes

17. Glossary